

2510-00093

Autonomous Defect Detection and Mapping in Pipeline Networks

Submitted by

RUBESH SURESH

Department of Mechanical Engineering

College of Design and Engineering

In partial fulfilment of the
requirements for the Degree of
Bachelor of Engineering
National University of Singapore

Session 2025/2026

Contents

Summary **iv**

Acknowledgement **v**

List of Figures **vi**

1 Introduction and Historical Background **1**

1.1 Purpose 1

1.2 Problem 2

1.3 Scope 2

2 Embedded System Design **4**

2.1 System Architecture Overview 4

2.2 Hardware Platform Selection 4

2.3 Sensor Suite 6

2.4 Communication and Data Pipeline 7

2.5 Design Limitations and Future Work 8

3 AI-Based Defect Detection Pipeline **9**

3.1 AI Pipeline Overview 9

3.2 Object Detection Architecture Selection 9

3.3 Dataset and Training Configuration 10

3.4 Training Results and Evaluation 12

3.5 Inference Pipeline and Edge Deployment 15

3.6 Sensor Integration and Event Logging 16

3.7 Limitations and Future Extensions 16

4 Visual SLAM for Pipeline Mapping 17

4.1 Overview and Project Context 17

4.2 Fundamentals of Visual SLAM 18

4.3 Conventional SLAM Systems versus the pySLAM Framework . . 20

4.4 Implementation Details 22

4.5 Future Extension: RGB-D Camera Integration 23

4.6 Summary 25

5 Conclusions and Recommendations for Future Work 27

5.1 Conclusions 27

5.2 Recommendations for Future Work 28

References 30

6 Appendix 33

6.1 Embedded System Hardware Prototype 33

6.2 Integrated Dashboard Screenshots 34

6.3 Project README 37

6.3.1 Overview 37

6.3.2 Project Structure 37

6.3.3 Quick Start on Raspberry Pi 5 39

6.3.4 How Each Subsystem Connects 40

6.3.5 Plugging In the Real Arduino JSON Stream 42

6.3.6 Swapping Between Live Webcam and Pre-recorded Video 42

6.3.7 Defect Event Recording 43

6.3.8 Attaching the Real SLAM Pipeline 44

6.3.9 Performance Notes for Raspberry Pi 5 45

6.3.10 Deployment Assumptions 46

6.3.11 API Reference 47
6.4 Python Dependencies (`requirements.txt`) 48
6.5 Runtime Configuration (`config.yaml`) 50

Summary

This project develops the sensing and intelligence components for an unmanned robot designed to inspect underground waste pipe networks in Singapores public housing estates. These Pneumatic Waste Conveyance Systems use high-speed air suction to transport refuse through buried steel pipes, but the pipes last only about 30 years before requiring costly replacement. Current inspection relies on workers physically entering confined underground spaces a hazardous and reactive process with no means of detecting problems early. This project supports a broader effort to extend pipe lifespan from 30 to 45 years through automated defect detection. Three core capabilities were developed. First, a compact hardware system combining two computing boards and six sensors monitors pipe conditions including temperature, distance, orientation, and air quality. Second, an AI model trained on over 22,000 labelled images identifies defects such as cracks, corrosion, and blockages from camera footage. Third, a spatial mapping system builds a 3D model of the pipe interior, allowing defects to be located on a map. All outputs feed into a live web dashboard. Testing confirmed end-to-end functionality on low-cost hardware without internet or expensive processors, though the AI model requires further training for reliable accuracy. Future priorities include extended training, upgrading to a depth-sensing camera, and adding dedicated AI processing hardware.

Acknowledgement

The Author would like to thank Prof. Chew Chee Meng for his guidance throughout the academic year, and acknowledge the use of Claude Opus 4.6 for the purposes of formatting, editing, and code generation for the dashboard prototype. Instructions for replicating the code dashboard, and some sample screenshots are given in the Appendix. The full code base can be found on GitHub here: https://github.com/sp4ce-cowboy/AddamNet

List of Figures

2.1 Embedded system architecture of the pipeline inspection platform. 5

3.1 Dataset statistics: class instance distribution (top-left), bounding box spatial distribution (bottom-left), and bounding box dimension distribution (bottom-right). 11

3.2 Sample mosaic-augmented training batch with ground-truth bounding box annotations overlaid. Class indices: 0–Deformation, 1–Obstacle, 2–Rupture, 3–Disconnect, 4–Misalignment, 5–Deposition. 12

3.3 Normalised confusion matrix after one training epoch. The *Disconnect* class achieves the highest true positive rate (0.65), while other classes exhibit high background misclassification. 13

3.4 Precision–Recall curves per class and overall (mAP@0.5 = 0.242). The *Disconnect* class (AP = 0.550) significantly outperforms other classes. 14

3.5 F1–Confidence curves per class. The optimal all-class F1 of 0.28 occurs at a confidence threshold of 0.169. 14
3.6 Validation batch predictions with predicted bounding boxes and confidence scores overlaid. 15

6.1 Embedded system angled view 34
6.2 Live Feed tab 35
6.3 Sensors tab 35
6.4 Defects tab 36
6.5 SLAM tab 36

Introduction and Historical Background

1.1 Purpose

The Pneumatic Waste Conveyance System (PWCS) is an automated underground pipe network that transports household waste from residential blocks to a centralised bin centre using high-speed air suction. In Singapore, the Housing and Development Board (HDB) has mandated that all new public housing projects with more than 500 dwelling units incorporate PWCS infrastructure. As of 2023, there are approximately 110 such systems across the island, with the majority located in newer precincts that are still under active development. The system eliminates the need for conventional truck-based refuse collection, reduces pest infestation, and improves the overall cleanliness of residential estates.

However, the underground mild steel pipes that form the backbone of PWCS are currently designed for a service life of roughly 30 years, after which they must be replaced through extensive and disruptive excavation works. This project contributes to a broader initiative by HDB to develop an unmanned inspection and maintenance robot capable of extending that lifespan from 30 to 45 years, by enabling early detection and in-situ repair of pipe defects such as corrosion, cracking, and wall thinning. Specifically, this work focuses on the sensing, perception, and intelligence layer of such a robot i.e. designing the embedded system, the artificial intelligence pipeline, and the spatial mapping framework that would allow a future robotic platform to autonomously inspect and diagnose PWCS pipe conditions.

1.2 Problem

Current inspection and repair of PWCS pipes is labour-intensive and hazardous. Workers must physically enter the underground pipeline network, a confined space with limited air supply potentially hazardous chemical exposure and pest exposure, to assess pipe integrity and clear blockages. At Punggol Northshore, for example, the pipe network serves over 5,300 units across 112 discharge valves, with suction distances of up to 4.08 km. Pipes have an outer diameter of 500 mm and wall thicknesses ranging from 7.9 mm at branch sections to 18 mm along main runs. Blockages, which are a frequent occurrence largely caused by residents disposing of prohibited items, are currently removed using a manually operated corkscrew device extended through manholes spaced every 40 metres; when the device fails, workers crawl into the pipe on roller planks to retrieve the obstruction by hand.

Beyond blockage removal, there is presently no systematic capability for predictive maintenance. Pipe defects such as internal corrosion and thinning particularly at bends, where special wear-resistant materials are already specified can progress undetected until a section must be condemned and replaced. This reactive approach results in extended system downtime and significant dis-amenity to residents.

1.3 Scope

This project addresses the sensor integration, on-board intelligence, and spatial mapping components required for an autonomous PWCS pipe inspection system. The work is divided into four main contributions:

1. **Embedded System Design.** A hardware prototype built around a Raspberry Pi 5 microprocessor paired with an Arduino Mega microcon-

troller. The Arduino interfaces with peripheral sensors for data acquisition, while the Raspberry Pi handles high-level processing and decision-making.

2. **AI-Based Defect Detection Pipeline.** A YOLOv8 object detection model trained on a curated dataset of pipe defects which includes corrosion patches, cracks, and regions of wall thinning, and optimised to run at the edge on the Raspberry Pi. The pipeline fuses visual detections with sensor readings forwarded from the Arduino to produce classified defect reports.

3. **Visual SLAM for Spatial Mapping.** A monocular Visual Simultaneous Localisation and Mapping (SLAM) implementation using pySLAM, capable of deriving depth information from standard 2D video without a dedicated depth sensor. This enables the system to map defect locations within a reconstructed three-dimensional model of the pipe network for diagnostic and predictive maintenance purposes.

4. **Integrated Monitoring Dashboard.** A real-time web-based dashboard built with FastAPI on the back end and Three.js on the front end, consolidating live sensor feeds, defect detection outputs, and 3D pipe reconstructions into a single operator interface.

Together, these components form an end-to-end proof-of-concept for the perception and intelligence subsystem of an unmanned PWCS inspection robot, laying the groundwork for future integration with a self-propelling mechanical platform.

Embedded System Design

2.1 System Architecture Overview

The embedded subsystem serves as the sensing and telemetry backbone of the autonomous pipeline inspection platform. It follows a two-tier distributed architecture. The lower tier, an Arduino Mega 2560, is responsible exclusively for sensor acquisition and serial transmission. The upper tier, a Raspberry Pi 5, handles all higher-level processing including defect detection inference, visual SLAM, and dashboard hosting. This separation of concerns ensures that time-sensitive sensor polling on the microcontroller is not disrupted by computationally expensive tasks.

Figure 2.1 illustrates the high-level data flow. The Arduino samples all active sensors at 10 Hz, encodes each measurement as a JSON packet, and transmits it over USB serial at 115 200 baud. The Raspberry Pi receives this stream via PySerial and routes it to the appropriate application modules. The JSON interface is deliberately standardised so that either tier can be upgraded independently, and additional microcontroller nodes can be added without restructuring the Pi-side software.

2.2 Hardware Platform Selection

The Arduino Mega 2560, based on the ATmega2560, was selected over the Uno and Nano on the basis of three critical requirements. First, its 256 KB flash and 8 KB SRAM [1] accommodate the sensor firmware, JSON formatting routines, and MPU-6050 calibration procedure that would exceed the Uno's

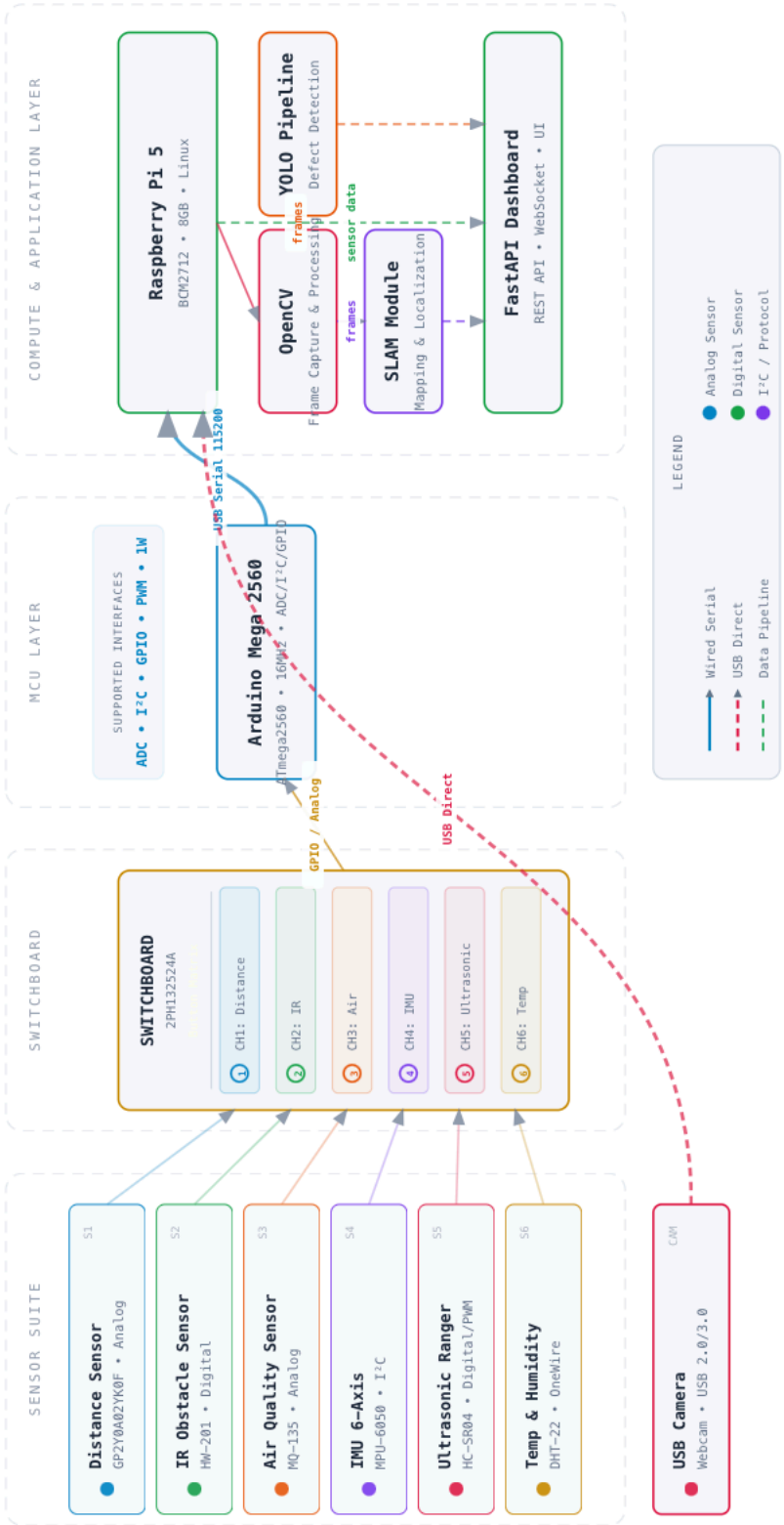


Figure 2.1: Embedded system architecture of the pipeline inspection platform.

32 KB flash and 2 KB SRAM. Second, its 54 digital I/O pins and 16 analogue inputs support the full six-sensor suite with spare capacity for future expansion. Third, four independent hardware UARTs (Universal Asynchronous Receiver-Transmitter) allow the USB serial link to the Raspberry Pi to operate without occupying the only available port, leaving capacity for future peripherals such as GPS.

The Raspberry Pi 5 (4 GB RAM) was selected as the edge computing host. Its quad-core Arm Cortex-A76 processor at 2.4 GHz provides sufficient throughput to run YOLO inference and a visual SLAM pipeline concurrently. The Debian-based Linux environment gives direct access to Python computer vision libraries including OpenCV, PyTorch, and PySerial, with no porting overhead. Its USB 3.0 ports simultaneously host the Arduino and the USB webcam without bandwidth contention.

2.3 Sensor Suite

Six sensors were selected to address environmental monitoring, proximity awareness, inertial localisation, and gas safety functions within a pipeline inspection context. Their specifications and roles are summarised in Table 2.1.

The MPU-6050 IMU communicates over I^2C at 400 kHz and provides the primary localisation input. A startup calibration routine averages 500 stationary readings to compute per-axis gyroscope and accelerometer bias offsets, which are subtracted from all subsequent measurements. Empirical testing identified a persistent gyroscope bias of approximately $-3.4\,^\circ/\text{s}$ on the Y axis; without correction, this would produce an orientation error exceeding $200\,^\circ$ per minute. Post-calibration residual drift was reduced to below $0.5\,^\circ/\text{s}$. The HC-SR04 and GP2Y0A02YK0F are used together to provide averaged forward distance from two independent physical principles, improving robustness against

Table 2.1: Sensor suite specifications and roles.

Sensor	**Range**	**Role in Pipeline Inspection**
DHT-22	−40 to +80 °C; 0 - 100% RH	Thermal and humidity anomaly monitoring
HC-SR04	2 - 400 cm	Forward obstacle detection; wall proximity
MPU-6050	±2 g; ±250 °/s	Inertial orientation and dead-reckoned displacement
MQ-135	Qualitative index	Hazardous gas detection (CO_2, NH_3, NOx, VOCs)
HW-201	2 - 40 cm	Short-range binary obstacle warning
GP2Y0A02YK0F	20 - 150 cm	Medium-range analogue IR distance measurement

single-sensor artefacts. The MQ-135 requires a 24 - 48 hour burn-in for stable baseline calibration; firmware outputs both a raw ADC value and a qualitative gas classification, permitting threshold refinement post-deployment.

2.4 Communication and Data Pipeline

USB-CDC serial at 115 200 baud was selected as the Arduino-to-Raspberry Pi interface. The connection requires no additional hardware, simultaneously powers the Arduino during bench operation, and is natively supported by PySerial. At 10 Hz with packets of approximately 380 bytes, link utilisation remains well below saturation.

Each packet is a single-line JSON object terminated by a newline. JSON was chosen because it is human-readable for debugging, self-describing (sensor fields can be added without modifying the parser), and maps directly to the FastAPI dashboard's WebSocket format. On the Arduino, the MPU-6050 is sampled on every loop iteration for maximum dead-reckoning resolution;

slower sensors are polled on dedicated software timers. On the Raspberry Pi, the PySerial receiver runs as a dedicated process and distributes parsed dictionaries to the YOLO, SLAM, and dashboard modules via shared memory queues. When a YOLO defect event is triggered, a snapshot of the current sensor dictionary is logged alongside the annotated frame, providing contextual metadata including estimated position, forward distance, temperature, and air quality for each detection.

The primary communication limitation is the absence of hardware timestamp synchronisation. Packet timestamps derive from the Arduino's `millis()` counter, which drifts independently of the Raspberry Pi system clock, precluding sub-millisecond sensor-camera temporal alignment without a shared hardware synchronisation signal.

2.5 Design Limitations and Future Work

Several limitations of the current implementation are acknowledged. Dead-reckoning from a consumer MEMS IMU accumulates integration error over time, and the MPU-6050 is not rated for navigation-grade accuracy and should be treated as a coarse displacement indicator. The monocular USB camera cannot resolve metric depth, constraining SLAM to scale-ambiguous maps. YOLO inference on the Raspberry Pi 5 CPU operates at reduced frame rates without hardware acceleration, limiting detection throughput.

Future extensions include a circumferential radial sensor array for full 360 degree pipe bore profiling, integration of a Hailo-8 AI accelerator for real-time inference [2], migration to a stereo or RGB-D camera to resolve depth ambiguity, and addition of a magnetometer fused with the IMU gyroscope to reduce long-run heading drift.

AI-Based Defect Detection Pipeline

3.1 AI Pipeline Overview

Manual visual inspection of pipeline infrastructure is labour-intensive, subjective, and constrained by the accessibility limitations of confined environments. The defect detection pipeline developed in this project addresses these limitations by employing a deep-learning-based object detection model to automatically localise and classify defects within video frames acquired during robotic inspection traversals. Within the broader system architecture, the detection pipeline operates alongside two concurrent perception subsystems, firstly the sensor telemetry acquisition via the Arduino subsystem and secondly visual Simultaneous Localisation and Mapping (SLAM), forming a multi-modal perception layer.

The pipeline is designed for edge deployment on a Raspberry Pi 5 platform, with all inference performed locally without reliance on cloud computing resources. When a defect is detected, a structured event record is generated containing the timestamp, detection confidence, detected class, frame reference, and a contemporaneous sensor snapshot from the Arduino subsystem. These records are streamed to a FastAPI-based monitoring dashboard for real-time operator awareness and stored for post-mission reporting.

3.2 Object Detection Architecture Selection

The You Only Look Once (YOLO) family of single-stage detectors was selected for this project owing to its favourable inference latency profile per-

forming localisation and classification in a single forward pass which is critical for resource-constrained deployment. Within the YOLOv8 family (Ultralytics, 2023) [3], the nano variant (YOLOv8n) was chosen as it offers the smallest parameter count (approximately 3.2 M parameters, 8.7 GFLOPs) and is therefore the most suitable variant for CPU-bound inference on the Raspberry Pi 5. While larger variants such as YOLOv8m or YOLOv8l may yield higher mean Average Precision (mAP) on benchmark datasets, their computational demand renders them impractical for embedded deployment without dedicated acceleration hardware. The selection of YOLOv8n represents a deliberate engineering tradeoff which is prioritising inference speed and deployment feasibility over marginal detection accuracy gains.

3.3 Dataset and Training Configuration

The training dataset comprises annotated pipeline inspection images labelled with bounding boxes in YOLO format across six defect classes: *Deformation*, *Obstacle*, *Rupture*, *Disconnect*, *Misalignment*, and *Deposition*. Figure 3.1 presents the class instance distribution, totalling approximately 22,279 annotated instances. The dataset exhibits moderate class imbalance, with *Obstacle* (5,912 instances) and *Deposition* (4,724) being the most represented classes, and *Disconnect* (1,948) the least.

Training was executed using the Ultralytics command-line interface with transfer learning from COCO-pretrained weights. The training configuration, as in the generated `args.yaml`, was as follows:

```
yolo detect train model=yolov8n.pt data=config.yaml \
    epochs=1 batch=32 imgsz=480 device=mps
```

An input resolution of 480 × 480 pixels was used, with a batch size of 32 and Apple Metal Performance Shaders (MPS) acceleration on the development

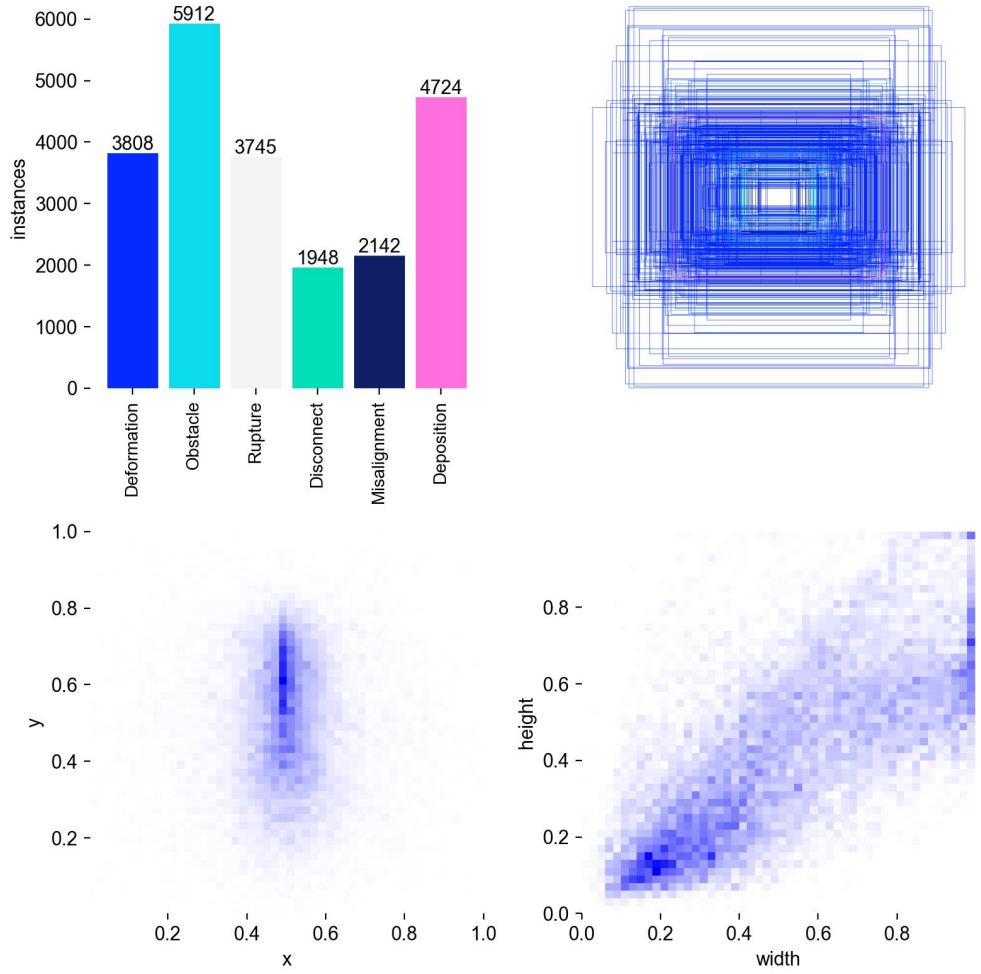


Figure 3.1: Dataset statistics: class instance distribution (top-left), bounding box spatial distribution (bottom-left), and bounding box dimension distribution (bottom-right).

machine. The current results reflect a single-epoch training run, serving as a proof-of-concept validation of the training pipeline. Figure 3.2 illustrates a representative mosaic-augmented training batch, demonstrating the diversity of pipeline inspection imagery and the applied data augmentation.

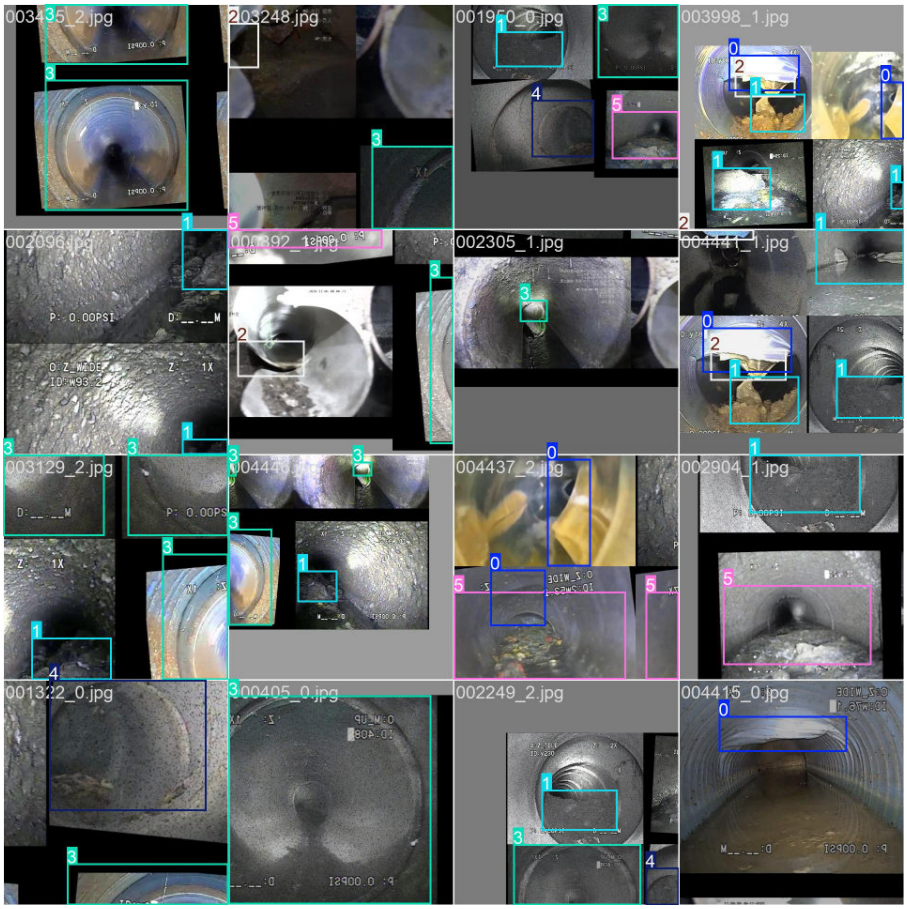


Figure 3.2: Sample mosaic-augmented training batch with ground-truth bounding box annotations overlaid. Class indices: 0–Deformation, 1–Obstacle, 2–Rupture, 3–Disconnect, 4–Misalignment, 5–Deposition.

3.4 Training Results and Evaluation

After one epoch of training, the model achieved a validation precision of 0.267, recall of 0.253, mAP@0.5 of 0.242, and mAP@0.5:0.95 of 0.081. These metrics are expected for a single-epoch baseline and indicate that the model has begun to learn discriminative features but has not yet converged. The training

and validation losses (box: 1.567/1.872, classification: 2.613/2.843, distribution focal: 1.645/1.922) confirm that further training epochs are required for convergence.

The normalised confusion matrix (Figure 3.3) reveals that the *Disconnect* class is detected most reliably (65% true positive rate), while *Deformation*, *Obstacle*, and *Rupture* are predominantly misclassified as background (81%, 86%, and 88% respectively). This indicates that the model has not yet learned sufficient feature representations for these classes after a single epoch.



Figure 3.3: Normalised confusion matrix after one training epoch. The *Disconnect* class achieves the highest true positive rate (0.65), while other classes exhibit high background misclassification.

The Precision–Recall curve (Figure 3.4) confirms that *Disconnect* achieves the highest per-class AP of 0.550, substantially outperforming the remaining classes. The F1–Confidence curve (Figure 3.5) indicates an optimal all-class F1 score of 0.28 at a confidence threshold of 0.169. This low optimal threshold reflects the model's current tendency toward low-confidence predictions, consistent with early-stage training.

Figure 3.6 presents validation batch predictions, showing that the model can localise several defect types particularly *Disconnect* and *Misalignment*

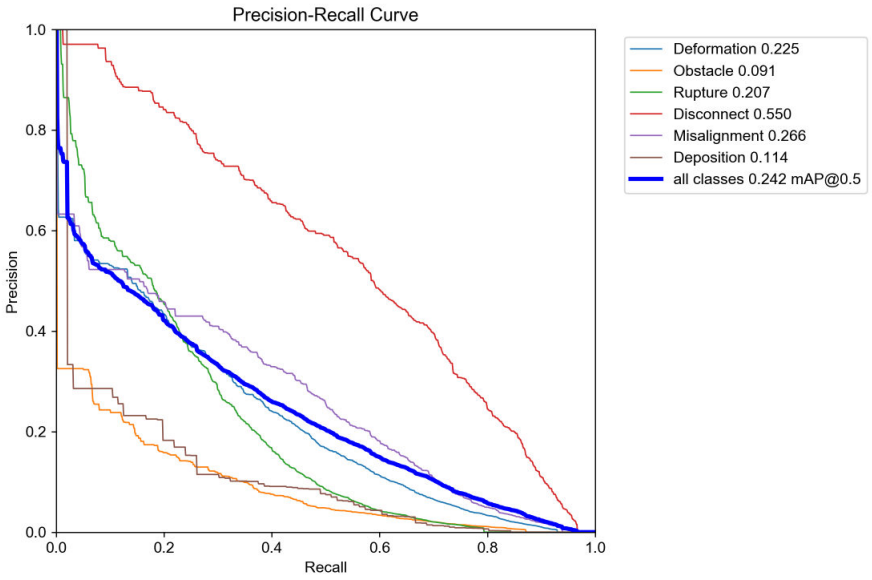


Figure 3.4: Precision–Recall curves per class and overall (mAP@0.5 = 0.242). The *Disconnect* class (AP = 0.550) significantly outperforms other classes.

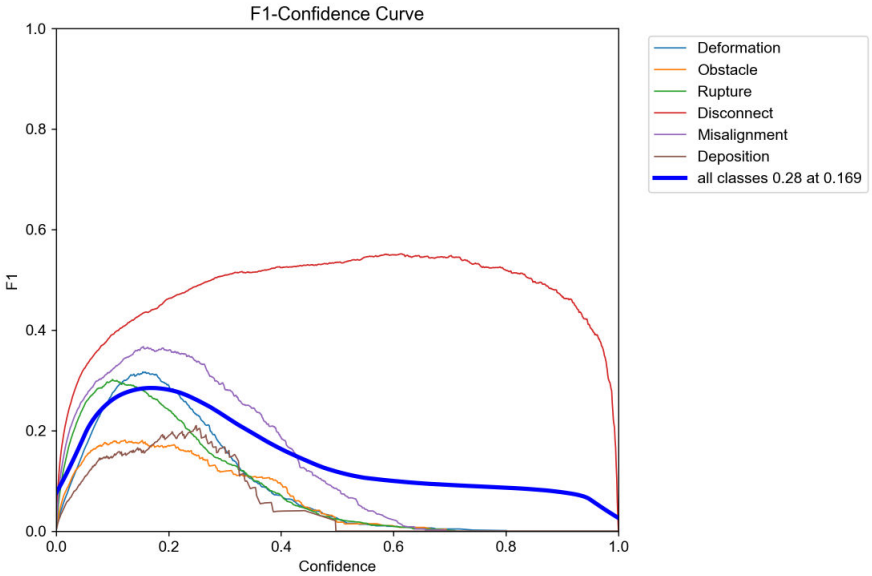


Figure 3.5: F1–Confidence curves per class. The optimal all-class F1 of 0.28 occurs at a confidence threshold of 0.169.

with reasonable spatial accuracy, though with frequent false negatives and low confidence scores at this training stage.

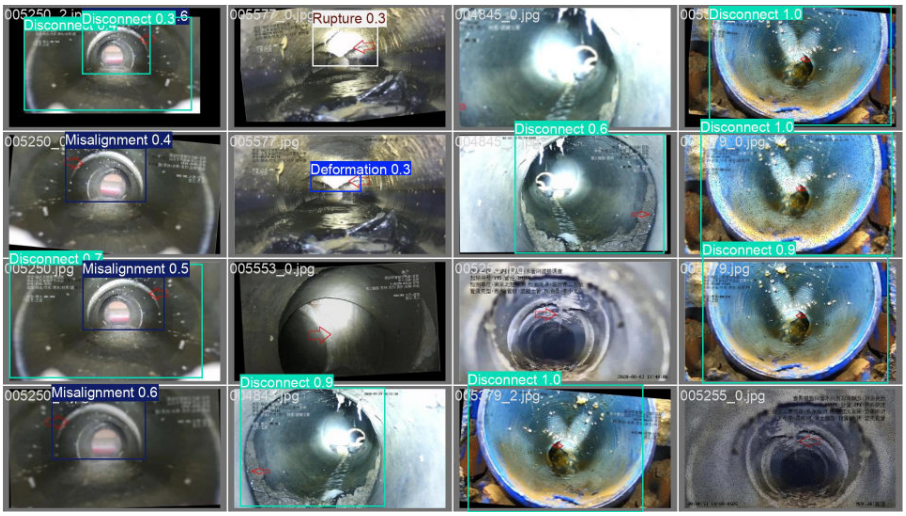


Figure 3.6: Validation batch predictions with predicted bounding boxes and confidence scores overlaid.

3.5 Inference Pipeline and Edge Deployment

The trained model checkpoint (`best.pt`) is deployed for inference on a frame-by-frame basis: each frame is extracted from the video source, resized to 480×480 pixels, passed through the YOLOv8n network, and filtered by a configurable confidence threshold. Surviving detections are rendered as annotated bounding boxes. The pipeline supports both offline video file input for post-mission analysis and live USB webcam input for real-time inspection.

On the Raspberry Pi 5, CPU-only inference with YOLOv8n is feasible at approximately 2–5 frames per second at 480×480 resolution. Frame skipping and resolution reduction strategies are employed to manage computational constraints. This constitutes a proof-of-concept validation that meaningful defect detection can be achieved on low-cost embedded hardware without GPU dependency.

3.6 Sensor Integration and Event Logging

Each detection event exceeding the confidence threshold triggers a contemporaneous sensor snapshot from the Arduino subsystem, capturing distance measurements, temperature, humidity, and IMU orientation data. These are appended to a structured defect event record containing the timestamp, confidence, class, frame reference, and sensor data. A cooldown mechanism prevents duplicate logging of the same defect across consecutive frames. The structured event log is streamed to the FastAPI dashboard, enabling real-time tabulated inspection reporting and operator alerting during active missions.

3.7 Limitations and Future Extensions

The current single-epoch training run yields preliminary metrics insufficient for production deployment, and extended training over 50–100 epochs is required to assess convergence behaviour and true model capability. The normalised confusion matrix indicates significant class confusion and high background false negative rates, particularly for visually subtle defect types. Additional limitations include monocular camera constraints precluding depth estimation, lighting variability in pipeline interiors, CPU inference throughput limitations, and the absence of inter-frame temporal tracking.

Future extensions include training with extended epoch counts and hyperparameter optimisation, deployment using the Hailo-8L AI accelerator or migration to an NVIDIA Jetson platform for GPU-accelerated inference, integration of detection outputs with the SLAM pipeline for spatially referenced defect mapping, multi-camera or depth-sensor fusion, and incorporation of a temporal tracking module such as ByteTrack [4] for persistent defect association across frames.

Visual SLAM for Pipeline Mapping

4.1 Overview and Project Context

Pneumatic waste conveyance systems rely on pressurised airflow to transport refuse through underground pipeline networks to a central collection facility, eliminating the need for conventional truck-based collection in dense urban areas. Over time, the interior surfaces of these pipelines are subject to wear, corrosion, sediment build-up, joint displacement, and structural deformation, any of which can degrade system throughput or trigger catastrophic failure [14]. Routine inspection is therefore essential, yet the enclosed, GPS-denied nature of the pipeline environment renders traditional surveying methods impractical.

Current practice relies heavily on closed-circuit television (CCTV) inspection, in which a tethered crawler robot transmits video footage to a surface operator who visually identifies defects [12, 13]. This workflow is slow, subjective, and incapable of producing a spatially registered record of defect locations within the network. An operator viewing a flat video stream cannot determine with certainty *where* along a branching pipeline a particular crack or deposit was observed, nor measure its true physical extent.

The objective of this project is to move beyond passive video inspection toward *autonomous defect detection with spatial awareness*. The enabling technology is Simultaneous Localisation and Mapping (SLAM), which allows the inspection robot to build a geometric map of the pipeline interior while simultaneously tracking its own position within that map [5]. When a defect is detected whether by a human reviewer or by a convolutional-neural-network classifier operating on the video frames, the SLAM-derived pose estimate pro-

vides the metric coordinates needed to locate, measure, and revisit the defect.

In the prototype implementation, a monocular camera mounted on the inspection platform captures first-person video as the robot traverses the pipeline. A lightweight visual SLAM pipeline, implemented entirely in Python with OpenCV and NumPy, processes this footage to produce three outputs: (i) a video overlay showing tracked features and an inlier count head-up display, (ii) a sparse three-dimensional point cloud of the pipeline interior, and (iii) an estimated six-degree-of-freedom camera trajectory. Post-processing reconstructs a surface mesh from the point cloud and renders it in an interactive browser-based viewer, providing the operator with a navigable 3D model of the inspected section.

4.2 Fundamentals of Visual SLAM

Visual SLAM addresses the chicken-and-egg problem of needing a map to localise the camera, yet needing the camera's location to build the map. The solution is to solve both problems simultaneously through an iterative, frame-by-frame pipeline whose core stages are feature extraction, data association, motion estimation, triangulation, and map maintenance [6].

Feature extraction and tracking. Each incoming frame is scanned for salient image patches. Classical detectors such as FAST (Features from Accelerated Segment Test) identify corner-like points by examining a ring of pixels around each candidate, while ORB (Oriented FAST and Rotated BRIEF) augments these detections with a rotation-invariant binary descriptor for matching [10]. Alternatively, features can be tracked across consecutive frames using the Kanade–Lucas–Tomasi (KLT) optical-flow algorithm, which models the local image motion as a translational shift and solves for it in a coarse-to-fine pyramid scheme. Our implementation defaults to KLT tracking, as it avoids

the per-frame cost of descriptor computation and is well-suited to the smooth, continuous motion typical of a pipeline crawler.

Motion estimation. Given a set of point correspondences between two frames, the relative camera rotation $\mathbf{R}$ and translation direction $\mathbf{t}$ are recovered via the essential matrix. The five-point algorithm, embedded within a RANSAC (Random Sample Consensus) loop, estimates $\mathbf{E}$ robustly in the presence of outlier matches [11]. Decomposition of $\mathbf{E}$ yields up to four candidate $(\mathbf{R}, \mathbf{t})$ solutions; the physically valid one is selected by verifying that triangulated points lie in front of both cameras. The translation vector is recoverable only up to an unknown scale factor in the monocular case, which is a fundamental limitation discussed further in Section 4.5.

Triangulation and mapping. Once the relative pose is known, inlier correspondences are triangulated into three-dimensional map points using the Direct Linear Transform (DLT) on the two camera projection matrices. Spurious points are filtered by enforcing positive depth in both camera coordinate frames, bounding the reprojection error, and discarding points at extreme distances. Over successive frames, the surviving map points accumulate into a sparse point cloud that captures the geometric skeleton of the scene.

Loop closure and global optimisation. A full SLAM system detects when the camera revisits a previously mapped region and introduces a constraint that corrects accumulated drift in both the trajectory and the map. Our lightweight prototype does not yet implement loop closure; drift therefore accumulates along long pipeline traversals. Production-grade systems such as ORB-SLAM3 [8] perform loop closure by matching a Bag-of-Words representation of the current frame against a database of keyframes, then running a pose-graph optimisation to distribute the accumulated error across the trajec-

tory.

4.3 Conventional SLAM Systems versus the pySLAM Framework

Established visual SLAM systems such as ORB-SLAM [6], ORB-SLAM2 [7], and ORB-SLAM3 [8] are implemented predominantly in C++ and are designed for real-time operation on desktop-class hardware. They comprise tightly coupled threads for tracking, local mapping, and loop closing, communicating through shared data structures protected by mutexes. While this architecture delivers state-of-the-art accuracy (ORB-SLAM3 reports average trajectory errors of 3.6 cm on the EuRoC drone benchmark and 9 mm in hand-held AR/VR sequences) it carries several practical drawbacks for a research-stage pipeline-inspection project.

First, the C++ codebase has deep dependencies on system-level libraries (Eigen, g2o, DBoW2, Pangolin, OpenCV compiled with specific flags), making compilation non-trivial on embedded platforms such as the Raspberry Pi. Second, the monolithic design makes it difficult to swap individual components, for example, to replace the ORB feature front-end with a learning-based detector, without substantial refactoring.

pySLAM, introduced by Freda [9], takes a fundamentally different architectural approach. It is a hybrid Python/C++ framework in which the high-level pipeline logic, configuration, and component orchestration are expressed in Python, while performance-critical inner loops (Hamming-distance matching, PnP solving, voxel hashing) are implemented in C++ and exposed through pybind11 bindings. This dual-language strategy permits rapid prototyping and component substitution in Python without sacrificing throughput in the numerical core.

The framework is explicitly modular. Local feature extraction is abstracted behind a unified interface that supports over thirty detector–descriptor combinations, from classical methods (ORB, SIFT, SURF, AKAZE, BRISK) to modern deep-learning-based alternatives (SuperPoint, KeyNet + AffNet + HardNet, LightGlue, XFeat). Loop closure is similarly pluggable, with backends ranging from traditional Bag-of-Words (DBoW2, iBoW) and VLAD to learned global descriptors such as NetVLAD, CosPlace, and EigenPlaces.

Beyond sparse SLAM, pySLAM integrates a volumetric reconstruction pipeline that consumes depth and colour images via Truncated Signed Distance Function (TSDF) integration with parallel voxel-block hashing, or alternatively through incremental Gaussian splatting [9]. It also embeds monocular depth-prediction models such as DepthPro, Depth Anything V2, and others directly into the SLAM loop, enabling pseudo-dense mapping even from a single camera. Semantic segmentation modules (DeepLabv3, Segformer, CLIP) can label map points with object categories, paving the way for semantic-aware defect mapping.

For the current project, pySLAM served as a conceptual and architectural reference rather than a direct runtime dependency. Our prototype reimplements the essential monocular visual odometry stages which are FAST detection, KLT tracking, essential-matrix estimation, and triangulation in a self-contained Python script with only OpenCV and NumPy as dependencies. This minimalist approach was chosen to ensure portability to the Raspberry Pi and to avoid the complex installation procedure of the full pySLAM environment. However, the modular design philosophy of pySLAM informed our code structure: the feature tracker, pose estimator, and map are encapsulated as separate classes, facilitating future upgrades such as plugging in a learning-based front-end or adding loop closure.

4.4 Implementation Details

The prototype pipeline operates in three stages. In the first stage, each video frame is optionally downscaled to 480p to reduce computational cost, converted to greyscale, and passed to the FAST corner detector. Detected keypoints are tracked to the subsequent frame via KLT optical flow using a three-level image pyramid and a 21×21 pixel search window. When the number of successfully tracked points drops below a threshold (400 points by default), a fresh detection is triggered to replenish the feature set.

In the second stage, the essential matrix is estimated from the surviving correspondences using OpenCV's five-point solver within a RANSAC framework ($p = 0.999$, threshold $= 1.0\,\text{px}$). The relative rotation and translation are recovered via `cv2.recoverPose`, and the global camera pose is updated by composition:

$$\mathbf{t}_k = \mathbf{t}_{k-1} + \mathbf{R}_{k-1}\,\mathbf{t}_{\text{rel}}, \qquad \mathbf{R}_k = \mathbf{R}_{\text{rel}}\,\mathbf{R}_{k-1}. \tag{4.1}$$

Inlier correspondences are triangulated into 3D map points using the DLT, filtered for positive depth and bounded reprojection error ($< 4.0\,\text{px}$), and appended to a global point-cloud buffer.

In the third stage, the accumulated point cloud and trajectory are exported as a PLY file and a CSV file respectively. A reconstruction script performs statistical outlier removal (rejecting points whose mean k-nearest-neighbour distance exceeds two standard deviations above the global mean), estimates surface normals via local PCA, and reconstructs a triangulated mesh using an alpha-shape algorithm with a k-nearest-neighbour connectivity fallback for sparse regions. The result is rendered as an interactive Three.js scene in a self-contained HTML file, offering orbit, pan, zoom, multiple colour modes, and

toggleable layers for the point cloud, mesh, wireframe, and camera trajectory.

4.5 Future Extension: RGB-D Camera Integration

The most significant limitation of the current monocular prototype is the absence of metric scale. Because a single camera cannot distinguish between a large, distant object and a small, nearby one, the translation vector recovered from the essential matrix is a unit vector: the trajectory and map are reconstructed up to an unknown global scale factor. In a pipeline of known nominal diameter this ambiguity can be partially resolved by fitting a cylinder to the point cloud and imposing the known radius, but this is fragile in the presence of deformations, junctions, and bends.

Upgrading to an RGB-D (colour-plus-depth) camera eliminates the scale ambiguity entirely. Each pixel in the depth image provides a direct metric measurement of the distance to the nearest surface, typically via structured-light projection or active stereo correlation. Cameras such as the Intel RealSense D435, which offers a depth range of up to 10 m with a global shutter and a compact 90 mm × 25 mm × 25 mm form factor, are well-suited to confined-space robotics [15]. For the particularly short working distances encountered inside small-diameter pipelines (200–600 mm), the RealSense D405 provides milimeter precision depth accuracy at ranges of 7–50 cm in a package smaller than a golf ball (42 mm × 42 mm × 23 mm) [16].

Integrating depth data into the SLAM pipeline yields improvements across every stage. At the front end, each detected feature can be immediately back-projected into 3D using the depth reading and the known camera intrinsics, bypassing the need for multi-view triangulation and thereby producing map points from the very first frame. This significantly accelerates initialisation, a

known weakness of monocular systems, which require sufficient translational parallax before any 3D structure can be recovered.

At the motion-estimation stage, the availability of 3D–2D correspondences enables Perspective-n-Point (PnP) solving rather than the epipolar-geometry-based essential-matrix approach. PnP with RANSAC is both faster and more robust, as it directly minimises reprojection error against known 3D landmarks, and it recovers the full six-degree-of-freedom pose including metric translation magnitude. ORB-SLAM3 exploits this strategy in its RGB-D mode, achieving an absolute trajectory error of 0.107 m on the TUM RGB-D benchmark in a comparative evaluation with RTAB-Map and OpenVSLAM [17].

For dense reconstruction of the path toward a navigable 3D model of the pipeline interior, depth images significantly improve the current system. Rather than relying on sparse triangulated points, the system can perform volumetric integration, fusing successive depth frames into a TSDF volume or a surfel-based model. pySLAM's volumetric module already supports this workflow using parallel block-based voxel hashing, and the resulting dense mesh captures not only the gross geometry of the pipe but also surface defects (cracks, corrosion pits, protruding joints) at the resolution of the depth sensor. Combined with the semantic segmentation models already available in pySLAM, each voxel could be labelled with a defect class, yielding a semantically annotated 3D defect map of the pipeline network.

A further practical advantage of RGB-D sensing is robustness in the low-texture conditions typical of pipeline interiors. Uniform concrete or PVC surfaces contain few distinctive visual features, causing both feature detection and KLT tracking to degrade. The depth channel provides geometric structure even in the absence of photometric texture, allowing the system to maintain tracking by aligning depth scans via Iterative Closest Point (ICP) algorithms. Hybrid systems that fuse both photometric and geometric residuals such as the

DVO-SLAM and ElasticFusion families have demonstrated superior robustness in precisely these conditions.

For deployment on the Raspberry Pi 5 or a comparable embedded platform, the Intel RealSense D400 series provides on-chip depth computation, offloading the stereo-matching workload from the host CPU and delivering aligned depth and colour frames at 30 fps over USB 3. The remaining SLAM computation (feature tracking, PnP, local bundle adjustment) is well within the capability of the Pi 5's quad-core Cortex-A76, particularly when operating at 480p resolution and leveraging the C++ core of pySLAM.

In summary, the transition from monocular to RGB-D sensing represents the single highest-impact upgrade path for the prototype. It resolves the metric-scale ambiguity, accelerates initialisation, enables dense volumetric mapping suitable for quantitative defect measurement, and improves robustness in the textureless environments characteristic of pneumatic waste conveyance pipelines.

4.6 Summary

This chapter has presented the visual SLAM subsystem of the autonomous pipeline inspection platform. Starting from the theoretical foundations of feature-based monocular SLAM, we described a lightweight prototype implementation that produces a sparse 3D map and camera trajectory from standard video footage. The modular architecture of the pySLAM framework was examined as a reference design, highlighting its advantages over monolithic C++ systems for research-stage projects that demand rapid component iteration. Finally, we outlined a concrete upgrade path centred on RGB-D camera integration, which would provide metric-scale mapping, dense reconstruction, and improved tracking robustness which are the key capabilities required to

advance from prototype to a deployable defect-mapping system for pneumatic waste conveyance infrastructure.

Conclusions and Recommendations for Future Work

5.1 Conclusions

This project developed a proof-of-concept perception and intelligence subsystem for an unmanned inspection robot targeting HDB's Pneumatic Waste Conveyance System pipeline network. The work addressed three core technical challenges which are embedded sensing, AI-based defect detection, and spatial mapping, and integrated their outputs through a real-time web-based monitoring dashboard.

A two-tier embedded architecture was implemented, pairing an Arduino Mega 2560 for deterministic sensor acquisition with a Raspberry Pi 5 for higher-level computation. Six sensors spanning environmental monitoring, proximity detection, inertial localisation, and gas safety were interfaced and sampled at 10 Hz, with data transmitted as standardised JSON packets over USB serial at 115 200 baud. A startup calibration routine for the MPU-6050 IMU reduced gyroscope bias drift from approximately 3.4 °/s to below 0.5 °/s, demonstrating that consumer-grade MEMS hardware can provide usable, if coarse, orientation data for dead-reckoned navigation inside GPS-denied pipeline environments.

A YOLOv8n object detection model was trained on a dataset of 22 279 annotated pipe defect instances across six classes and deployed for CPU-only inference on the Raspberry Pi 5 at 2–5 fps. Although the single-epoch baseline yielded modest validation metrics (mAP@0.5 of 0.242), the pipeline successfully demonstrated end-to-end functionality: frames were captured, prepro-

cessed, classified, and forwarded together with contemporaneous sensor snapshots to the FastAPI dashboard for real-time operator review. The result validates the feasibility of meaningful edge-based defect detection on low-cost embedded hardware without cloud or GPU dependency.

A monocular visual SLAM pipeline was implemented using FAST feature detection, KLT optical-flow tracking, essential-matrix estimation, and DLT triangulation. The system produced a sparse three-dimensional point cloud and a six-degree-of-freedom camera trajectory from standard video footage, subsequently reconstructed into an interactive Three.js mesh viewer. While the monocular approach inherently lacks metric scale, the prototype confirmed that spatial mapping of pipeline interiors is achievable with minimal hardware.

Collectively, these contributions establish a functional perception layer that can be integrated with a future self-propelling mechanical platform, advancing HDB's goal of extending PWCS pipe service life from 30 to 45 years through predictive maintenance.

5.2 Recommendations for Future Work

Extended training of the YOLOv8n model over 50–100 epochs with hyperparameter tuning and class-balancing strategies is the most immediate priority, as current metrics reflect an unconverged baseline. Deployment of a dedicated AI accelerator such as the Hailo-8L, or migration to an NVIDIA Jetson platform, would increase inference throughput to real-time rates and enable larger model variants.

Upgrading the monocular camera to an Intel RealSense D405 RGB-D sensor would resolve the metric-scale ambiguity, enable dense volumetric reconstruction via TSDF integration, and improve tracking robustness in the low-texture pipeline interior. Integration of loop-closure detection would further

reduce trajectory drift over long inspection traversals.

On the embedded side, fusion of a magnetometer with the MPU-6050 gyroscope would mitigate long-run heading drift, while a circumferential radial sensor array would enable full 360 degree bore profiling for quantitative wall-thickness assessment. Finally, incorporating a temporal tracking module such as ByteTrack would allow persistent defect association across frames, reducing duplicate detections and enabling defect growth monitoring over successive inspection missions.

References

[1] Arduino S.r.l., "Arduino Mega 2560 Rev3 Datasheet," 2023. [Online]. Available: `https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf`

[2] Hailo Technologies Ltd., "Hailo-8 AI Processor Datasheet," 2024. [Online]. Available: `https://hailo.ai/products/ai-accelerators/`

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[4] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022.

[5] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.

[6] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[7] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[8] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for vi-

sual, visual-inertial and multi-map SLAM," *IEEE Trans. Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[9] L. Freda, "pySLAM: An open-source, modular, and extensible framework for SLAM," *arXiv preprint arXiv:2502.11955*, 2025. [Online]. Available: `https://github.com/luigifreda/pyslam`

[10] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2011, pp. 2564–2571.

[11] D. Nistér, "An efficient solution to the five-point relative pose problem," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004.

[12] J. C. P. Cheng and M. Wang, "Automated detection of sewer pipe defects in closed-circuit television images using deep learning techniques," *Automation in Construction*, vol. 95, pp. 155–171, 2018.

[13] S. I. Hassan *et al.*, "Underground sewer pipe condition assessment based on convolutional neural networks," *Automation in Construction*, vol. 106, p. 102849, 2019.

[14] T.-Y. Chuang and C.-C. Sung, "Learning and SLAM based decision support platform for sewer inspection," *Remote Sensing*, vol. 12, no. 6, p. 968, 2020.

[15] L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, and A. Bhowmik, "Intel RealSense stereoscopic depth cameras," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops*, 2017, pp. 1–10.

[16] Intel RealSense, "Depth Camera D405 – Product Specification," 2024. [Online]. Available: https://www.intelrealsense.com/depth-camera-d405/

[17] A. Ahangar *et al.*, "Comparative evaluation of RGB-D SLAM methods for humanoid robot localization and mapping," *arXiv preprint arXiv:2401.02816*, 2024.

Appendix

This appendix collates supporting material for the implementation chapters: screen captures of each dashboard tab, the project's consolidated technical `README`, the full Python dependency manifest, and the runtime configuration file used during the final demonstration. All source code, assets and configuration files referenced in this chapter are hosted in the public repository at `https://github.com/sp4ce-cowboy/AddamNet`.

6.1 Embedded System Hardware Prototype

The embedded system consolidates compute, sensing and human-interface components onto a single mounting board. The Raspberry Pi 5 runs the dashboard, YOLO inference and SLAM; the Arduino Mega 2560 handles sensor acquisition and streams JSON telemetry to the Pi over USB serial at 115200 baud. Figure 6.1 shows it at an oblique angle with the camera in frame.

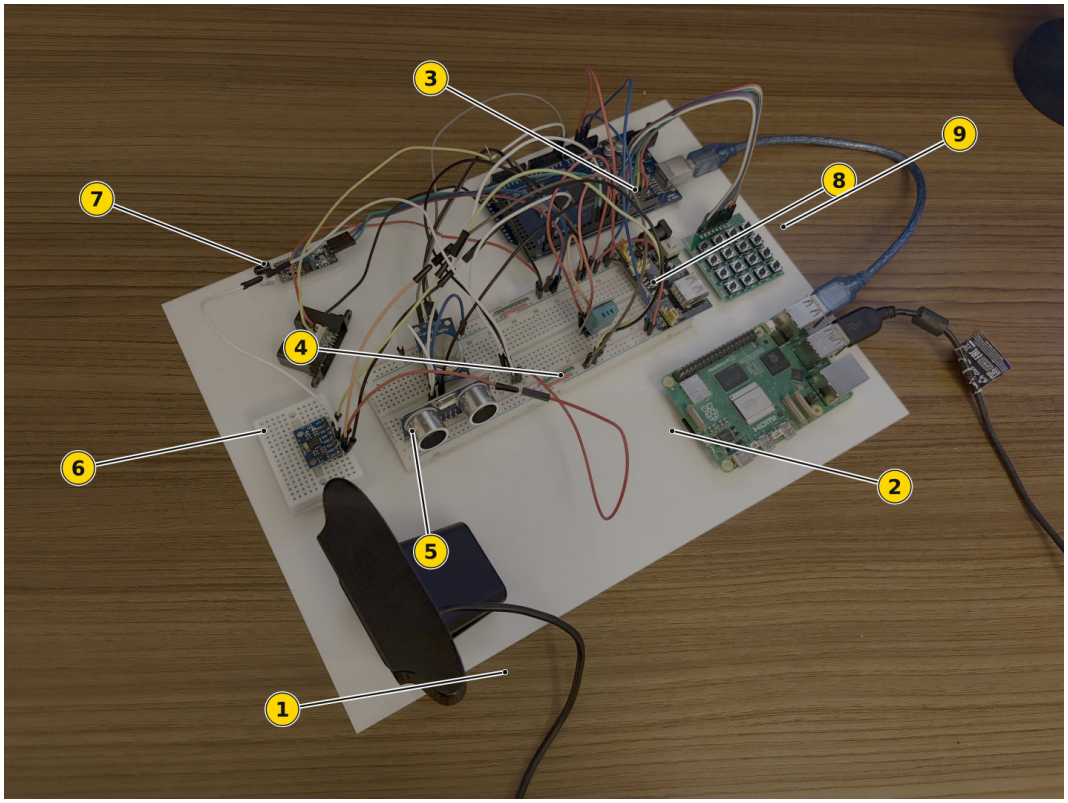


Figure 6.1: **Embedded system, angled view.** (1) Logitech UVC USB webcam (front-mounted); (2) Raspberry Pi 5; (3) Arduino Mega 2560; (4) Main breadboard; (5) HC-SR04 ultrasonic ranger; (6) MPU6050 inertial measurement unit; (7) HW201 IR Obstacle Sensor (8) DHT11 environmental module; (9) 4×4 matrix keypad;

6.2 Integrated Dashboard Screenshots

The following figures document the final state of the five-tab dashboard as deployed on the Raspberry Pi 5. Each figure corresponds to a single tab of the unified web interface served over the local network at port 8000.

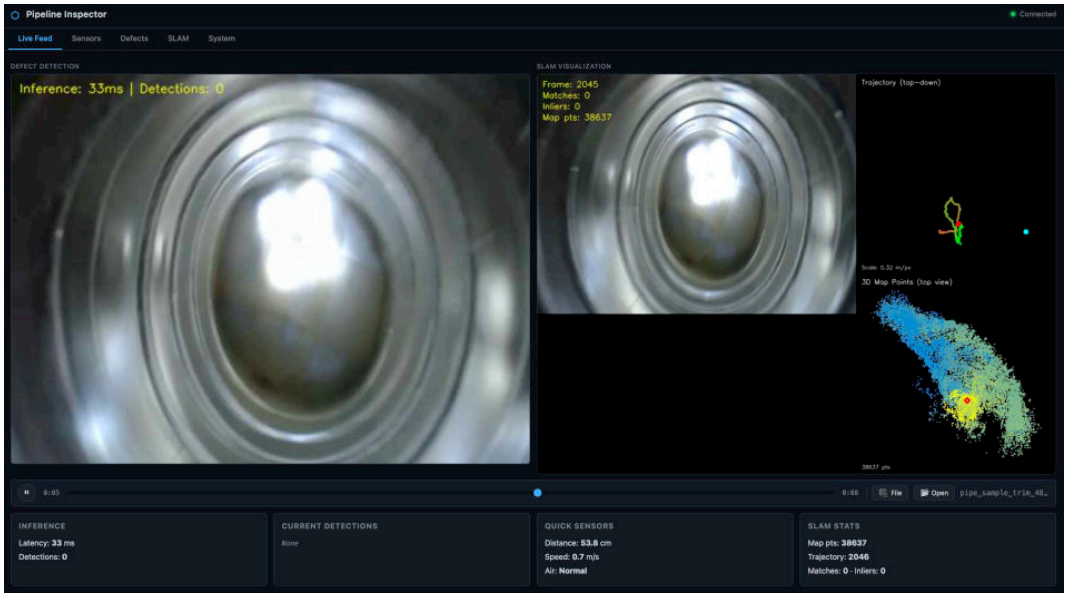


Figure 6.2: **Live Feed tab.** Left panel shows the YOLO defect detection stream with per-frame inference latency and detection count overlaid. Right panel shows the live SLAM visualisation, with the current frame, tracking statistics (matches, inliers, map points) and side-by-side top-down trajectory and 3D map-point projections. The bottom summary cards provide at-a-glance values for inference, current detections, quick sensor readings and SLAM statistics.

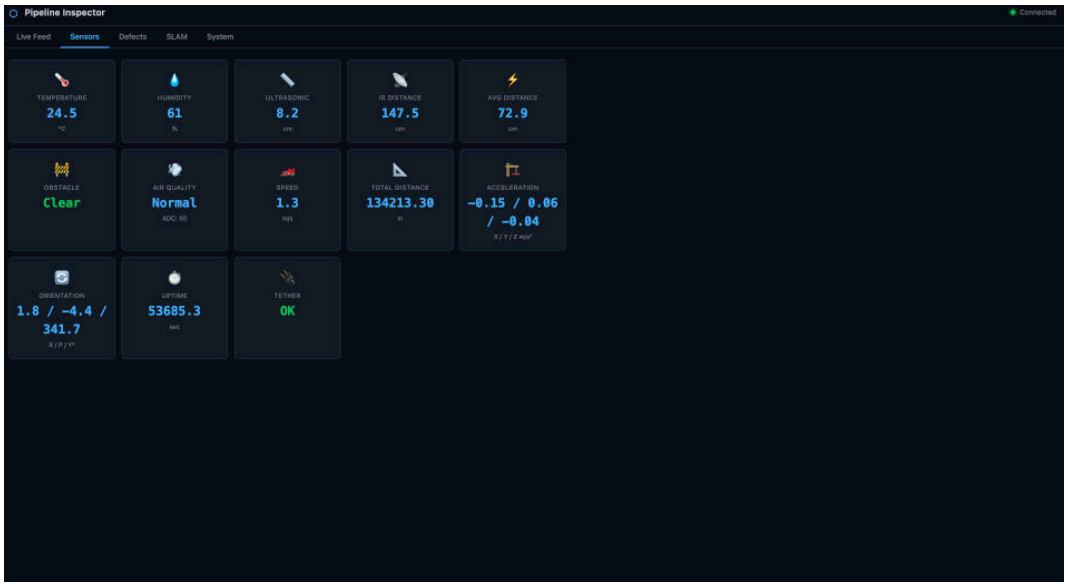


Figure 6.3: **Sensors tab.** Real-time telemetry pushed over the WebSocket from the Arduino Mega sensor suite. Tiles cover environmental readings (temperature, humidity, air quality), ranging (ultrasonic and IR distance with fused average), obstacle state, kinematic state (speed, odometer, acceleration, orientation) and system health (uptime and tether status).

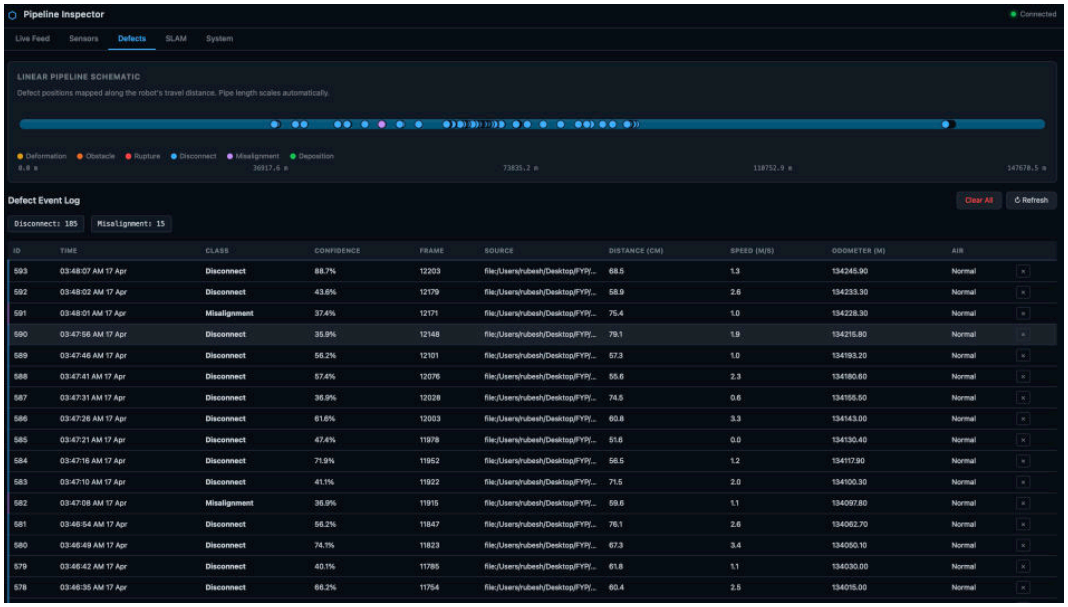


Figure 6.4: **Defects tab.** Upper section renders a linear pipeline schematic where each logged defect is placed along the robot's travel distance, with a legend colour-coding the six defect classes. The lower section is the full event log, showing per-event timestamp, class, confidence, frame index, source identifier, distance, speed, odometer reading, and the air-quality category at the moment of detection.

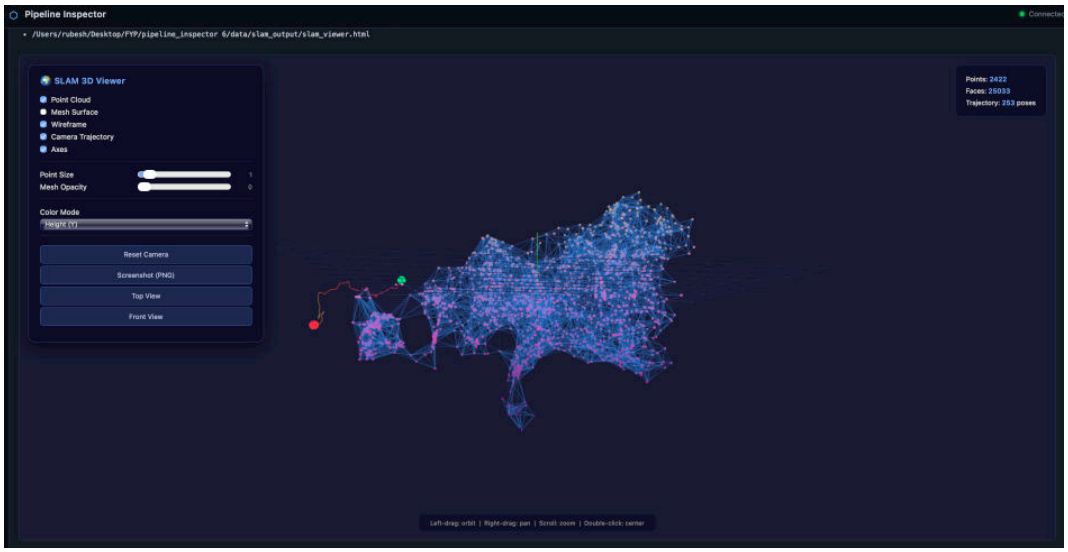


Figure 6.5: **SLAM tab.** Interactive 3D reconstruction viewer generated by the offline SLAM pipeline. The control panel toggles point cloud, mesh surface, wireframe, camera trajectory and world axes, with adjustable point size, mesh opacity, and height- or intensity-based colour modes. The displayed reconstruction contains 2,422 points, 25,033 mesh faces and a 253-pose camera trajectory.

6.3 Project README

The following is the complete `README.md` that accompanies the project repository. It serves as the canonical developer-facing reference for installation, configuration, subsystem integration and performance tuning on the Raspberry Pi 5 target platform. The original Markdown file is also included within the dashboard.

6.3.1 Overview

Integrated dashboard for autonomous pipeline inspection which combines embedded sensing, AI defect detection, and visual SLAM into a single Raspberry Pi 5 application built with FastAPI primarily in Python with some embedded Three.js for the SLAM viewing.

6.3.2 Project Structure

```
pipeline_inspector/
|-- run.py                      <- Entry point: starts the dashboard server
|-- config.yaml                 <- Central configuration (edit this)
|-- requirements.txt            <- Python dependencies
|-- README.md
|
|-- app/
|   |-- __init__.py
|   |-- main.py                 <- FastAPI app: wires services, lifespan
|   |-- config.py               <- YAML config loader
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   `-- schemas.py          <- Pydantic models (SensorSnapshot, etc.)
|   |
```

```
|   |-- services/
|   |   |-- __init__.py
|   |   |-- sensor_service.py <- Arduino serial reader (background)
|   |   |-- video_service.py  <- Webcam / video file capture (background)
|   |   |-- yolo_service.py   <- YOLO inference + annotation
|   |   |-- defect_logger.py  <- SQLite defect event storage with cooldown
|   |   `-- slam_service.py   <- SLAM pipeline wrapper
|   |
|   |-- api/
|   |   |-- __init__.py
|   |   `-- routes.py             <- REST + WebSocket endpoints
|   |
|   `-- static/
|       |-- index.html            <- Dashboard UI (single-page, 5 tabs)
|       |-- style.css             <- Dark industrial theme
|       `-- app.js                <- WebSocket client + DOM updates
|
|-- slam/
|   |-- __init__.py
|   |-- visual_slam.py            <- Existing SLAM code
|   |-- reconstruct_viewer.py     <- Existing 3D viewer generator
|   `-- pipeline.py               <- Existing pipeline runner
|
|-- weights/
|   `-- best.pt                   <- Trained YOLO model weights
|
|-- arduino/
|   |-- sensor_suite.ino            <- Arduino sketch (reference copy)
|   `-- sample_sensor_readings.txt  <- Sample JSON output for reference
|
`-- data/
    |-- defects.db                <- SQLite database (auto-created)
    `-- slam_output/              <- SLAM output directory (auto-created)
```

6.3.3 Quick Start on Raspberry Pi 5

1. Install dependencies

```
# Option A: System-wide (simpler)
pip install -r requirements.txt --break-system-packages

# Option B: Virtual environment (cleaner)
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2. Edit `config.yaml`

Open `config.yaml` and adjust for the target setup:

```
video:
  source: "webcam"           # "webcam" for live camera, "file"
     for pre-recorded
  webcam_device: 0            # usually 0 for /dev/video0
  video_file: "/path/to/your/video.mp4"  # used when source is
     "file"
  target_fps: 5               # lower = less CPU usage

serial:
  port: "/dev/ttyUSB0"        # or "auto" to detect, or "mock"
     for fake data
```

3. Run

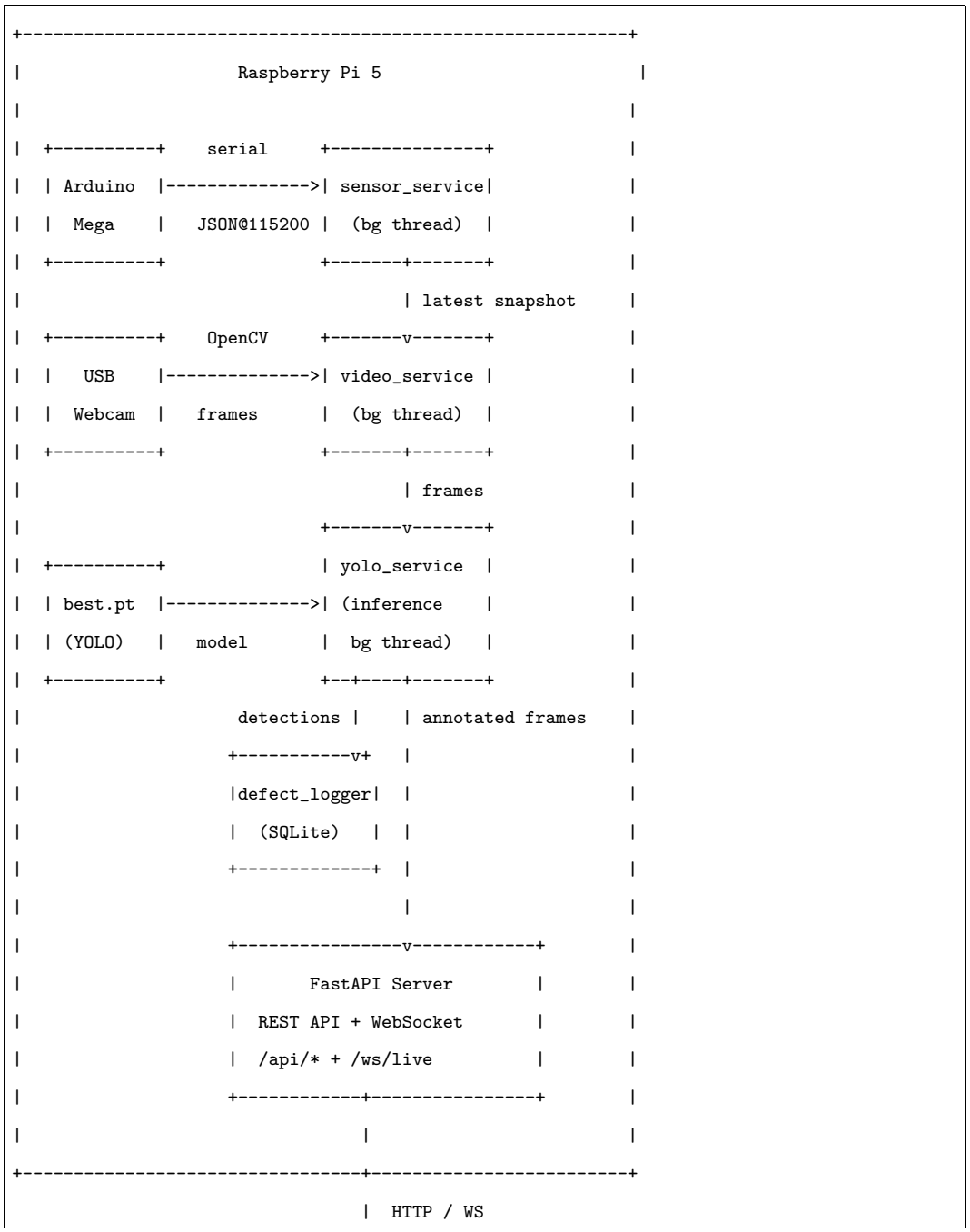
```
cd pipeline_inspector
python3 run.py
```

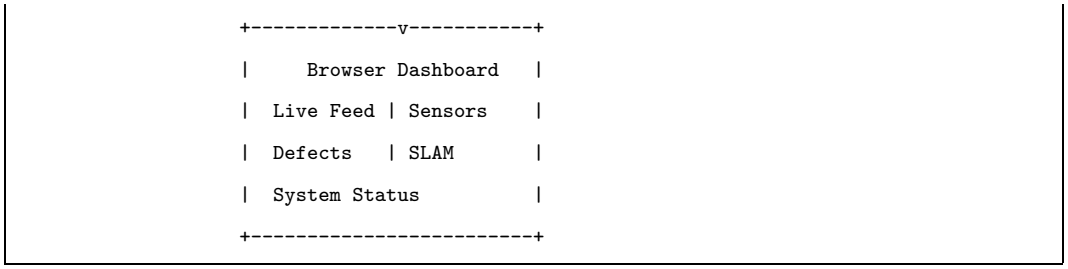
4. Open the dashboard

Navigate to `http://<raspberry-pi-ip>:8000` or the IP configured in `config.yaml` in any browser on the same network.

6.3.4 How Each Subsystem Connects

Architecture Diagram





Data Flow

1. **sensor_service** runs a background thread that reads JSON lines from the Arduino Mega's serial port. Each JSON line is parsed into a `SensorSnapshot` and stored in memory as `sensor_service.latest`. The WebSocket pushes this to the frontend at ∼4 Hz.

2. **video_service** runs a background thread that captures frames from either the USB webcam (`cv2.VideoCapture(0)`) or a video file. Frames are downscaled and rate-limited to the configured `target_fps`. The latest frame is stored thread-safely.

3. **yolo_service** loads `best.pt` at startup. A separate inference loop in `main.py` pulls the latest frame from `video_service`, runs YOLO detection, draws bounding boxes, and stores the annotated frame. The WebSocket sends the annotated frame as base64 JPEG.

4. **defect_logger** is called whenever YOLO produces detections. It applies a per-class cooldown (default 5 seconds) to avoid spamming duplicate events, then inserts a `DefectEvent` into the SQLite database along with the current sensor snapshot.

5. **slam_service** wraps the existing `visual_slam.py` and `reconstruct_viewer.py`. It can run the full SLAM pipeline on a video file in a background thread, tracking progress. The generated HTML viewer is served at `/slam-viewer`.

6.3.5 Plugging In the Real Arduino JSON Stream

The Arduino Mega sends JSON at 115200 baud over USB serial. The format used in `sample_sensor_readings.txt` is fully supported.

Steps:

1. Plug the Arduino Mega into the Raspberry Pi via USB.
2. Find the port:

```
ls /dev/ttyUSB* /dev/ttyACM*
```

   It will typically be `/dev/ttyUSB0` or `/dev/ttyACM0`.

3. Set the port in `config.yaml`:

```
serial:
  port: "/dev/ttyUSB0"    # or "auto" to auto-detect
```

4. If permission errors occur:

```
sudo usermod -aG dialout $USER
# then log out and back in
```

5. The dashboard will immediately start showing live sensor values.

Mock mode: If the Arduino is not connected and `mock_fallback: true` is set (the default), the system generates realistic synthetic sensor data so the dashboard still functions for development and demonstration.

6.3.6 Swapping Between Live Webcam and Pre-recorded Video

Edit `config.yaml`:

```yaml
video:
  # For live USB webcam:
  source: "webcam"
  webcam_device: 0

  # For pre-recorded video:
  source: "file"
  video_file: "/home/pi/videos/sewage.mp4"
```

The same YOLO inference pipeline runs on both sources. When using a video file, it loops automatically when it reaches the end.

Fallback: If `source: "webcam"` but the webcam is unavailable, and a `video_file` path is configured, the system automatically falls back to the video file.

6.3.7 Defect Event Recording

When YOLO detects one or more objects in a frame:

1. Each detection has a `class_name` and `confidence` score.
2. The **cooldown mechanism** checks whether this class has been logged in the last N seconds (configurable via `defects.cooldown_seconds`, default 5 s, but can be changed in `config.yaml`). If so, the detection is skipped. This prevents the log from being flooded with duplicate entries when the camera sees the same defect across many consecutive frames.
3. If the cooldown has elapsed, a `DefectEvent` is created containing:
   - Timestamp
   - Defect class name (Deformation, Obstacle, Rupture, Disconnect, Misalignment, or Deposition)

- Confidence score
- Frame number
- Video source identifier
- **Current sensor snapshot** (temperature, humidity, distance readings, speed, odometer, air quality) — this is the key integration point between the AI and embedded subsystems.

4. The event is inserted into the SQLite database at `data/defects.db`. The initial SQLite database is populated with some sample data, this can be cleared in the Application through the `Defects` tab.

5. The Defects tab on the dashboard shows all events in a sortable table, with per-class statistics and colour-coded rows.

6.3.8 Attaching the Real SLAM Pipeline

The SLAM integration is designed as a practical integration point that can be upgraded incrementally.

Current capability (already working)

- The SLAM tab shows pipeline status and output files.
- A SLAM run can be triggered on any video file from the dashboard.
- The run executes `visual_slam.py` followed by `reconstruct_viewer.py` in a background thread with progress tracking.
- The generated 3D HTML viewer is served directly at `/slam-viewer`.

To run SLAM

1. Go to the SLAM tab in the dashboard.

2. Enter the full path to a video file on the Pi.

3. Click "Run SLAM".

4. Watch the progress bar. When complete, click "Open 3D Viewer".

Alternatively via the API:

```
curl -X POST
   "http://localhost:8000/api/slam/run?video_path=/home/pi/videos/pipe.mp4"
```

Future upgrade path to semi-live SLAM

The `slam_service.py` module is structured such that the following extensions can be added:

- A method that processes frames incrementally (from `video_service`).
- Status updates pushed via the existing WebSocket.
- Periodic regeneration of the HTML viewer.

The SLAM modules in `slam/` are the original files, completely unchanged. Any improvements made to them will automatically be used by the dashboard.

6.3.9 Performance Notes for Raspberry Pi 5

Expected bottlenecks

1. **YOLO inference** — the largest CPU consumer. At `imgsz=320` and `target_fps=5`, expect ~200–400 ms per frame on the Pi 5 CPU. This is manageable at 2–5 FPS.

2. **Video capture** — OpenCV's `VideoCapture` is lightweight. The `target_fps` setting skips frames so the camera does not pile up a buffer.

3. **WebSocket streaming** — Base64 JPEG encoding adds ∼5–10 ms per frame. The JPEG quality is set to 45 for the WebSocket to minimise bandwidth.

4. **SLAM processing** — offline (runs on pre-recorded video), so it does not compete with live inference. It will take several minutes for a typical video.

Tuning for performance

- Lower `target_fps` in the config (3 FPS is still very usable).
- Lower `yolo.imgsz` to 256 (faster inference, slightly less accuracy).
- Lower `video.frame_width` / `frame_height` to 480 × 360.
- The sensor service is very lightweight (∼1 Hz serial reads).

Memory

On a 4 GB Pi 5, the YOLO model (∼6 MB) plus OpenCV frames plus the web server should comfortably fit within 1–2 GB. The remaining memory is available for SLAM processing when triggered.

6.3.10 Deployment Assumptions

1. **Raspberry Pi 5 (4 GB)** running Raspberry Pi OS (Bookworm, 64-bit).
2. **Python 3.11+** available system-wide.
3. **USB webcam** at `/dev/video0` (Logitech or any webcam that supports USB Video Class (UVC)). If absent, mock or file mode is used.
4. **Arduino Mega** at `/dev/ttyUSB0` sending JSON at 115200 baud. If absent, mock sensor data is generated.

5. **No GPU** — YOLO runs on CPU. The `imgsz=320` setting keeps inference manageable on the Pi's ARM Cortex-A76 cores.

6. **Network access** — the dashboard is served on port 8000 and is accessible from any device on the same LAN.

7. **SLAM is offline** — runs on pre-recorded video, not the live feed. This is by design, to respect the Pi's resource constraints.

6.3.11 API Reference

Method	Endpoint	Description
GET	`/`	Dashboard HTML
GET	`/api/sensors`	Latest sensor snapshot (JSON)
GET	`/api/defects?limit=N`	Recent defect events (JSON array)
GET	`/api/status`	System status summary
GET	`/api/slam`	SLAM pipeline status
POST	`/api/slam/run?video_path=..`	Trigger SLAM run
GET	`/api/feed/frame`	Latest annotated JPEG frame
GET	`/api/detections`	Current frame detections
GET	`/slam-viewer`	Serve generated SLAM 3D HTML viewer
WS	`/ws/live`	Real-time sensor + frame + detection stream

6.4 Python Dependencies (`requirements.txt`)

The `requirements.txt` file below pins the Python packages required to run the dashboard on the Raspberry Pi 5. It covers the web framework and data-validation layer, the serial-communication stack for the Arduino sensor suite, the computer vision and YOLO inference dependencies, and the numerical libraries used by the SLAM pipeline.

```
# ============================================================
#  Pipeline Inspection Dashboard - Python Dependencies
# ============================================================
#  Install on Raspberry Pi 5:
#    pip install -r requirements.txt --break-system-packages
#
#  Or in a virtual environment:
#    python3 -m venv venv && source venv/bin/activate
#    pip install -r requirements.txt
# ============================================================

# -- Web framework --
fastapi>=0.110.0
uvicorn[standard]>=0.29.0
websockets>=12.0

# -- Data validation --
pydantic>=2.6.0

# -- Config --
pyyaml>=6.0

# -- Serial communication (Arduino) --
pyserial>=3.5

```

```
# -- Computer vision --
opencv-python-headless>=4.9.0

# -- AI / YOLO --
ultralytics>=8.1.0

# -- SLAM dependencies --
numpy>=1.26.0
scipy>=1.12.0
python-multipart
```

Listing 6.1: Project `requirements.txt`.

6.5 Runtime Configuration (`config.yaml`)

The `config.yaml` file below is the single source of runtime configuration consumed by `app/config.py` at application start-up. It controls the video source and frame geometry, YOLO model path and inference thresholds, serial port parameters for the Arduino sensor link, the defect event-logging database path and per-class cooldown, the SLAM output directory and live-visualisation toggles, and the FastAPI server bind address.

```yaml
# ============================================================
#  Pipeline Inspector - Configuration
# ============================================================
#  Copy this file and edit for the Raspberry Pi setup.
#  All paths are relative to the project root unless absolute.
# ============================================================

# -- Video / Camera Source --
video:
  # "webcam" or "file"
  source: "webcam"
  # webcam device index or /dev/videoN path
  webcam_device: 0
  # path to pre-recorded video (used when source == "file")
  video_file: ""
  # target processing FPS (lower = less CPU on RPi)
  target_fps: 5
  # frame width for processing (downscale from camera)
  frame_width: 640
  frame_height: 480

# -- YOLO Defect Detection --
yolo:
  model_path: "weights/best.pt"
```

```yaml
  confidence_threshold: 0.35
  # inference size (smaller = faster on RPi)
  imgsz: 320
  # class names mapped by index
  class_names:
    0: "Deformation"
    1: "Obstacle"
    2: "Rupture"
    3: "Disconnect"
    4: "Misalignment"
    5: "Deposition"

# -- Serial / Arduino Sensors --
serial:
  # "auto", a device path like "/dev/ttyUSB0", or "mock"
  port: "/dev/ttyUSB0"        # on macOS, check with: ls -a
     /dev/tty*
  baud_rate: 115200
  timeout: 2
  # if true, generate fake sensor data when serial unavailable
  mock_fallback: true

# -- Defect Logging --
defects:
  db_path: "data/defects.db"
  # seconds of cooldown per class before logging again
  cooldown_seconds: 5.0

# -- SLAM --
slam:
  output_dir: "data/slam_output"
  # latest HTML viewer path (auto-populated after a run)
  viewer_html: ""
  # Live SLAM visualization on the Live Feed tab
```

```
  live_enabled: true
  # Process every Nth frame (higher = less CPU, 1 = every
      frame)
  live_process_every_n: 2

# -- Server --
server:
  host: "127.0.0.1"
  port: 8000
```

Listing 6.2: Project `config.yaml`.